IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 6 June 2025, accepted 17 June 2025, date of publication 23 June 2025, date of current version 30 June 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3582523

RESEARCH ARTICLE

Benchmarking Model Predictive Control and Reinforcement Learning-Based Control for Legged Robot Locomotion in MuJoCo Simulation

SHIVAYOGI AKKI, (Student Member, IEEE), AND TAN CHEN, (Member, IEEE)
Electrical and Computer Engineering Department, Michigan Technological University, Houghton, MI 49931 USA

Corresponding author: Tan Chen (tanchen@mtu.edu)

This work was supported in part by U.S. National Science Foundation under Grant IIS-2348323.

ABSTRACT Model Predictive Control (MPC) and Reinforcement Learning (RL) are two prominent strategies for controlling legged robots. RL learns control policies through system interaction, adapting to various scenarios, whereas MPC relies on a predefined mathematical model to solve optimization problems in real-time. Despite their widespread use, there is a lack of direct comparative analysis under standardized conditions. This work addresses this gap by benchmarking MPC and RL controllers on a Unitree Go1 quadruped robot within the MuJoCo simulation environment, focusing on a standardized task, straight walking at a constant velocity. Performance is evaluated based on disturbance rejection, energy efficiency, and terrain adaptability. The results show that RL excels in handling disturbances and maintaining energy efficiency but struggles with generalization to new terrains due to its dependence on learned policies tailored to specific environments. In contrast, MPC shows enhanced recovery capabilities from larger perturbations by leveraging its optimization-based approach, allowing for a balanced distribution of control efforts across the robot's joints. The results present the advantages and limitations of both RL and MPC, offering insights into selecting an appropriate control strategy for legged robotic applications.

INDEX TERMS Controllers, legged locomotion, model predictive control, MuJoCo simulation, proximal policy optimization, reinforcement learning, robotics.

I. INTRODUCTION

In the rapidly evolving field of robotics, legged robots stand out for their ability to navigate unstructured terrains where wheeled robots typically struggle [1]. Applications such as disaster response and exploration benefit from the adaptability of legged locomotion [2], [3]. Early legged robots featured simple control systems like open-loop feedforward control that permitted limited mobility [4]. Over the past few decades, advancements in technology have led to the development of more complex legged robots, featuring multiple joints and limbs that require seamless coordination to maintain balance and achieve locomotion on varied surfaces [5], [6]. This complexity has necessitated the adoption of advanced control strategies. Initially, predefined gaits and closed-loop feedback mechanisms, such as Proportional-Derivative (PD) controllers, were used to achieve locomotion. However, these traditional controllers lack the adaptability needed for more complex and unpredictable environments [7]. Consequently, more sophisticated methods have been developed, including Central Pattern Generators (CPGs), Model Predictive Control (MPC), and Reinforcement Learning (RL).

CPGs generate adaptive and synchronized limb movements by mimicking the rhythmic patterns found in biological systems. By adjusting CPG parameters and integrating with sensory feedback, the coupled oscillator models create stable rhythmic patterns that can be modulated to produce various gaits such as walking, trotting, and galloping [8]. MPC is an advanced control strategy that optimizes future control actions by predicting the system's behavior over a defined horizon. It continuously solves an optimization problem at each time step, incorporating system constraints

The associate editor coordinating the review of this manuscript and approving it for publication was Martin Reisslein.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

to determine the effective control inputs [9]. RL has been used to train locomotion policies on quadrupedal robots, enabling them to adapt to various terrains through interaction with the environment and receiving feedback on their performance [10].

Given the diverse applications of these control strategies, it is evident that selecting an appropriate control method is crucial for optimizing legged locomotion performance. Currently, the two most prominent control strategies in legged locomotion are RL and MPC [11], [12], [13], [14]. Despite extensive research in both MPC and RL, there is a lack of a direct comparative analysis evaluating their performance in legged robot locomotion under consistent experimental conditions. In contrast, the work in [15] compares RL and optimal control (OC) for autonomous drone racing and shows that RL outperforms OC because RL can optimize a better objective. This paper offers a structured simulation-based comparison using the Unitree Go1 robot, focusing on a standardized walking task [16]. Through this setup, we highlight the practical differences in performance, robustness, and energy efficiency of each method. Additionally, identifying the limitations of each method provides valuable insights that can help integrate them with other classical control approaches to improve the performance of legged locomotion. Rather than providing new theoretical insights, this work offers more practical value through its application focus.

Because of the structural similarities between Go1 and other quadrupedal robots such as Boston Dynamics' Spot and MIT's Cheetah, the controllers discussed in this paper can be adapted to other robotic platforms, making the work useful to researchers working with diverse legged robots. Note that the MPC requires a model while the RL is entirely model-free. The performance comparison is inherently complex, as each method has distinct advantages and limitations. To address this, this paper focuses on a fundamental and standardized locomotion task to highlight the differences and illustrate their characteristics in a simulated environment facilitated by MuJoCo. This approach also ensures a fair evaluation by minimizing the influence of other factors, with a focus on the comparison of control performance. Section II provides a review of the work on RL, MPC, and the combined controllers with some examples. Section III outlines the problem formulation and the algorithm setup. Section IV describes the experimental results and further discussion in Section V. Finally, Section VI concludes this work and suggests directions for future research.

II. RELATED WORK

This section reviews the work on RL, MPC, and combination methods for legged locomotion. It is worth noting that a direct comparative analysis between RL and MPC under a standardized task and controlled environment has not been conducted. This paper addresses this gap by systematically evaluating the strengths and limitations of RL and MPC controllers for legged locomotion.

A. REINFORCEMENT LEARNING

Reinforcement Learning [17] is a machine learning approach where an agent learns to make decisions by interacting with an environment to maximize cumulative rewards. Early RL methods, like Q-learning [18], evolved to Deep Reinforcement Learning (DRL) [19], [20] with the integration of neural networks, enabling complex problem-solving. Modern algorithms such as Proximal Policy Optimization (PPO) [21] and Soft Actor-Critic (SAC) [22] address stability and convergence issues, extending RL's applicability to high-dimensional, continuous environments. Figure 1 provides a more detailed classification of RL algorithms [23]. RL has been applied to robotics for tasks like locomotion [24], [25], manipulation [26], [27], and navigation in uncertain environments. It has also been used in autonomous vehicles for decision-making and path planning, leveraging its ability to learn from interaction and adapt in real time.

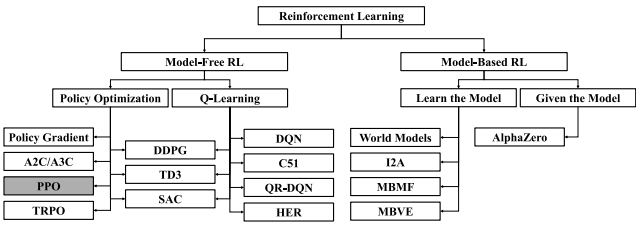


FIGURE 1. RL algorithm taxonomy. This paper adopts the highlighted PPO algorithm, one of the most widely used approaches.

B. MODEL PREDICTIVE CONTROL

Model Predictive Control is an advanced control strategy that optimizes control actions by predicting future system behavior over a finite horizon using a mathematical model. It has become popular for controlling multivariable systems with constraints due to its ability to handle dynamics in real time. Some excellent review articles were published, focusing on both its theoretical aspects [28], [29], [30] and its practical applications [31]. Recent developments in MPC focus on enhancing computational efficiency, robustness, and adaptability. Aligned with advancements in computational power and the availability of complex process models for various systems, MPC has enabled control over systems that were previously considered unmanageable. Its applications have extended beyond traditional chemical processes to diverse fields, including autonomous aerial vehicles [32] and legged robotics [33], [34], [35].

C. COMBINATION OF RL AND MODEL-BASED CONTROL

Recent research has explored various strategies to combine these approaches effectively [36], [37]. One method involves using RL to optimize the parameters of MPC, enhancing its performance and adaptability. For instance, [38] proposed a framework where RL adjusts MPC parameters to improve locomotion performance and stability in quadruped robots. Another approach employs MPC as a supervisory controller to ensure safety and stability, while RL handles tasks requiring adaptability. Reference [39] combined MPC with RL to

achieve agile and robust quadruped locomotion, effectively managing the trade-off between computational complexity and real-time control. These studies underscore the potential of combining RL and MPC to develop control systems that are both robust and adaptable, capable of operating effectively in dynamic and uncertain environments. Some research has also been conducted to address the limitations of RL by combining it with other methods to improve the computational efficiency or performance [40], [41].

D. OUR CONTRIBUTIONS

Our contributions can be summarized as follows.

- This is the first study that directly compares Model Predictive Control (MPC) and Reinforcement Learning (RL) for legged robot locomotion under controlled simulation environments. The comparison is performed using the Unitree Go1 quadruped robot within the MuJoCo simulation framework, focusing on a basic straight walking task. By maintaining consistency in the environment, task specifications, and robot configuration, this study ensures an unbiased evaluation of both methods.
- We assess the performance of each controller with respect to several quantifiable metrics, including disturbance rejection ability, energy efficiency, and recovery under failure. These performance measures offer insights into the trade-offs involved when choosing between MPC and RL for real-world robotic locomotion.
- To maintain transparency and reproducibility, we use a robust simulation setup incorporating tools such as OpenAI Gym, RL Baselines3 Zoo, MJPC, and MuJoCo. Code for our method, our implementation of the baseline, and all the data sets generated and used in this paper are released as open source. This can provide practical guidance for entry-level engineers and researchers in legged locomotion.

III. METHODS

The primary objective of this work is to benchmark the performance of MPC and RL controllers in a simulated environment. The benchmarking is conducted on a straight walking task at a constant velocity. The task performance is evaluated based on the disturbance rejection ability, the success of recovery under failure, and the energy efficiency. We utilize the Unitree Go1 robot, which is recognized for its reliability and advanced locomotion capabilities in robotics research [16]. The simulation is conducted in MuJoCo (Multi-Joint Dynamics with Contact), which is a high-performance physics engine designed for simulating complex dynamic systems, particularly in robotics and biomechanics [42], [43], [44]. Recent work focuses on integrating MuJoCo with high-level libraries and frameworks like OpenAI Gym and RL Baselines, enhancing its capabilities for end-to-end training pipelines in robotics [45].

This section elaborates on the methods for comparing the two controllers. It includes the simulation environment setup for both MPC and RL along with a presentation of the adopted algorithms. Data processing and analysis are performed in MATLAB. The data and simulation videos are available on GitHub [46].

A. RL ALGORITHM AND SETUP

The progression of different RL algorithms from value-based methods to policy optimization is illustrated in [23]. One notable advancement in this transition is Trust Region Policy Optimization (TRPO), which improves policy gradient methods by ensuring monotonic improvement through constrained optimization [47]. TRPO achieves this by introducing the concept of a trust region, which limits the magnitude of policy updates to prevent overly aggressive changes.

The trust region is defined by a constraint on the Kullback-Leibler (KL) divergence between the current policy and the updated policy. This ensures that updates remain within a predefined region where the approximation used in policy optimization remains valid. By adhering to this constraint, TRPO enhances the stability and reliability of policy gradient methods over episodes, preventing performance degradation caused by abrupt or large changes to the policy. TRPO's main contribution lies in its focus on improving the stability and reliability of policy gradient methods by ensuring that each policy update does not significantly deviate from the previous one, thereby theoretically guaranteeing monotonic improvement.

The optimization problem for TRPO written in terms of expectation is:

$$\max_{\theta} \mathbb{E}_{s,a\sim\pi_{\text{old}}}\left[\frac{\pi_\theta}{\pi_{\text{old}}}A_{\pi_{\text{old}}}\right], \tag{1}$$

subject to:

$$\mathbb{E}_{s\sim\pi_{\text{old}}}\left[D_{\text{KL}}(\pi_{\text{old}}\|\pi_\theta)\right] \le \delta, \tag{2}$$

where $\mathbb{E}$ represents the expected value operator, π_θ is the policy parameterized by θ, generating action a in state s, $A_{\pi_{\text{old}}}$ is the advantage function under the old policy, D_{KL} is the KL divergence, and δ is a small positive number dictating the size of the trust region. The advantage function $A_{\pi_{\text{old}}}$ represents the difference between the expected return under policy π_θ and the average expected return under the old policy π_{old}.

In this research, we adopt an actor-critic type Proximal Policy Optimization (PPO) algorithm for benchmarking (see Figure 2). Building on TRPO, PPO simplifies TRPO's approach to maintain a trust region by using a clipped objective function, making it easier to implement and scale [21]. It retains the benefits of TRPO but with less computational complexity. The objective function for PPO is:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_{s,a\sim\pi_{\text{old}}}\Big[\min(\frac{\pi_\theta}{\pi_{\text{old}}}A_{\pi_{\text{old}}}, \text{clip}(\frac{\pi_\theta}{\pi_{\text{old}}}, 1-\epsilon, 1+\epsilon)A_{\pi_{\text{old}}})\Big], \tag{3}$$

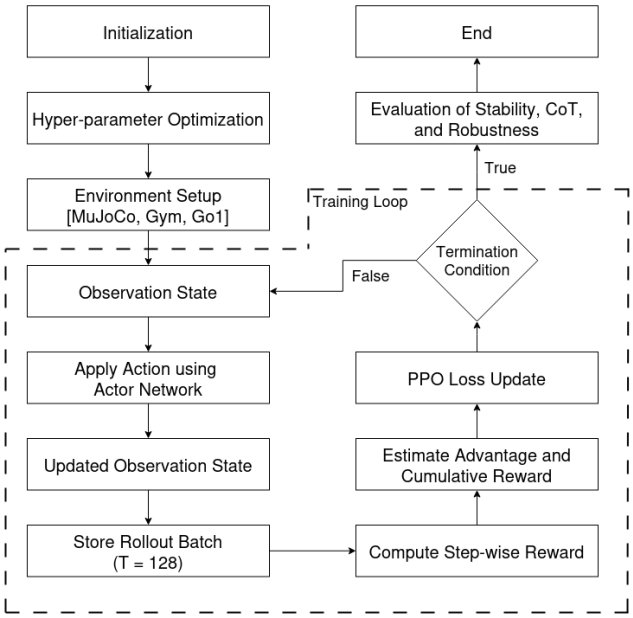


FIGURE 2. Reinforcement learning training procedure with proximal policy optimization algorithm.

where ϵ is a hyperparameter (typically small, e.g., 0.1 or 0.2) that controls the extent to which the policy can change in a single update (the "clipping" parameter). This objective encourages improving the policy π_θ as long as the ratio between the new and old policies remains within the interval $[1-\epsilon, 1+\epsilon]$, effectively constraining the policy update within a "proximal" region around the old policy.

1) ALGORITHM SELECTION JUSTIFICATION

PPO was chosen over alternatives like SAC or TD3 based on systematic comparisons for quadruped locomotion [48]. As shown in Table 1, PPO outperforms SAC and TD3 in simpler state spaces (Configs v0-v2) similar to our Go1 setup. For our standardized straight-walking task (requiring only body orientation and actuator states), PPO provides optimal balance of:

- *Stability*: Robust convergence with minimal hyperparameter tuning
- *Efficiency*: Faster training than SAC/TD3 for structured tasks
- *Simplicity*: Lower implementation complexity than off-policy methods

While SAC excels in high-dimensional sensor fusion (Configs v5-v6), our task requires only the v2 subset where PPO demonstrates superior performance.

The RL training environment for the Go1 robot is set up using the OpenAI Gym API [49], with the MuJoCo physics engine handling simulation and RL Baselines3 Zoo (rlzoo3) [50] used for training the PPO agent. A custom environment is created to define the observation and action spaces, reward function, and termination conditions.

TABLE 1. RL algorithm performance across sensor configurations [48].

Config	Sensor Inputs	Best Algorithm	Key Characteristics
v0	Body orientation only	PPO	Only PPO achieved stable gait
v1	v0 + Joint positions	PPO	PPO remained superior
v2	v1 + Joint velocities	PPO	Fastest convergence for Go1-relevant tasks
v3	v2 + Joint torques	TD3	Marginally better but slower convergence
v4	v3 + Foot sensors	SAC	Better for rough terrain
v5	v4 + IMU accelerometers	SAC	Required for complex maneuvers
v6	v5 + IMU gyroscopes	SAC	Full sensor fusion
Go1 Task	*Body orientation, joint states*	**PPO (v2)**	Matches flat-terrain requirements

2) REWARD FUNCTION FORMULATION

The reward function is carefully designed to incentivize stable and efficient locomotion:

$$r = w_f \cdot R_{\text{vel}} + w_h \cdot R_{\text{height}} + R_{\text{lateral}} + R_{\text{orientation}} - w_c \cdot \|a\|^2, \quad (4)$$

where w_f, w_h, and w_c are weighting coefficients, and a is the action vector (applied joint torques). The reward function combines five components to incentivize stable, efficient locomotion: (1) A forward velocity reward (R_{vel}) promotes maintaining the target speed of 0.5 m/s through asymmetric rewards that favor forward progress while penalizing stalling; (2) A height reward (R_{height}) maintains optimal center-of-mass height (0.26 m) via strong penalties for vertical deviations; (3) A lateral drift penalty (R_{lateral}) discourages sideways movement from the intended path; (4) An orientation penalty ($R_{\text{orientation}}$) stabilizes base posture by penalizing deviations from upright orientation; and (5) A control cost ($-w_c \cdot \|a\|^2$) promotes energy efficiency by penalizing large joint torque magnitudes, where a is action. Weighted by coefficients $w_f = 2.0$, $w_h = 1.2$, and $w_c = 0.05$, this formulation balances precise velocity tracking with postural stability and energy efficiency.

Before training, hyperparameter optimization is done using Optuna [51], including learning rate, number of steps per rollout, batch size, and clip range, based on cumulative reward performance. Once the best hyperparameters are found, the PPO algorithm from rlzoo3 is trained over a total of 1 million timesteps using the selected settings. During training, the agent collected rollouts consisting of observations, actions, rewards, and next states. These were used to compute advantage estimates through a learned value function, and the policy was updated using PPO's clipped surrogate loss to ensure stable learning. This cycle of collecting rollouts and

TABLE 2. List of key simulation parameters.

No.	Parameter	Value
1	Robot total mass	12 kg
2	Simulation timestep	0.001 s
3	Simulation duration per trial	50 s
4	Target velocity	0.5 m/s (along x-axis)
5	RL algorithm	PPO (Proximal Policy Optimization)
6	RL training timesteps	1 million
7	RL policy architecture	MlpPolicy (fully connected)
8	PPO learning rate	5.9e-4
9	PPO rollout steps per update	128
10	PPO clip range	0.2
11	Hyperparameter tuning method	Optuna
12	MPC controller type	MJPC Predictive Sampling
13	Perturbation magnitude	150 N (for standard tests)
14	Perturbation duration	0.2 s
15	Terrain friction (normal)	0.6 (sliding), 0.3 (torsional), 0.3 (rolling)
16	Terrain friction (slippery)	0.1 (sliding), 0.1 (torsional), 0.1 (rolling)
17	CoT computation duration	50 s

updating the policy was repeated until training was complete. The final trained policy was then evaluated across multiple episodes to benchmark performance in terms of locomotion stability, energy efficiency, and disturbance recovery. The key parameters are listed in Table 2.

B. MPC ALGORITHM AND SETUP

Building on the fundamental principles of MPC, the predictive sampling algorithm in the MuJoCo MPC (MJPC) framework can simplify real-time predictive control for complex robotics tasks [52]. In this paper, we utilize the predictive sampling algorithm. The process begins with the system's initial conditions including actuator type and joint states, followed by state estimation as shown in Figure 3. The optimization problem is in the following form:

$$\min_{s_{1:T}, a_{1:T}} \sum_{t=0}^{T} c(s_t, a_t), \tag{5}$$

subject to:

$$s_{t+1} = m(s_t, a_t), \text{ given } s_0, \tag{6}$$

where the objective is to optimize state $s \in \mathbf{S}$ and action $a \in \mathbf{A}$ trajectories, governed by a model $m : \mathbf{S} \times \mathbf{A} \to \mathbf{S}$, which defines the system's dynamics. The minimization goal is to compute the total return, represented as the sum of step-wise values $c : \mathbf{S} \times \mathbf{A} \to \mathbb{R}_+$, along the trajectory from the current state to the horizon T:

$$c(s, a) = \sum_{i=1}^{N} w_i \cdot \mathrm{n}_i\big(\mathbf{r}_i(s, a)\big). \tag{7}$$

The cost function (Equation 7) is expressed as the sum of N terms, where each term consists of three key components. First, it includes a nonnegative scalar weight $w \geq 0$ that defines the relative importance of the term within the overall cost. Second, it incorporates a norm function $\mathrm{n}(\cdot)$, which operates on a vector and produces a nonnegative scalar value, achieving its minimum when the vector equals zero. Finally, the residual, denoted as $\mathbf{r}(s, a)$, represents a measure of deviation or error in the system's state s and action a relative to the desired task or target.

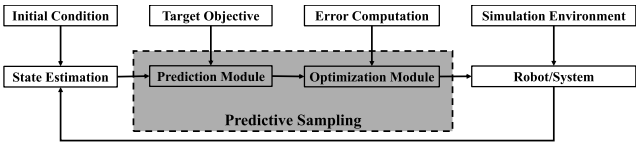


FIGURE 3. Model predictive control framework using predictive sampling.

1) SPECIFIC RESIDUAL FORMULATION

For our MPC implementation using the MJPC framework, we define four key residuals that encode the locomotion objectives:

$$\mathbf{r}_0 : \text{Vertical torso stability relative to foot height} \tag{8}$$

$$\mathbf{r}_1 : \text{Goal position deviation } \|q_t - q_t^d\| \tag{9}$$

$$\mathbf{r}_2 : \text{Desired torso orientation deviation } \|o_t - o_g\| \tag{10}$$

$$\mathbf{r}_3 : \text{Control effort } \|u_t\| \tag{11}$$

where q_t is current position, q_t^d is desired position, o_t is current orientation, and o_g is goal orientation. The complete cost function is then:

$$c(s, a) = \sum_{i=0}^{3} w_i \cdot \|\mathbf{r}_i(s, a)\|^2 \tag{12}$$

with squared Euclidean norms ($\mathrm{n}_i(\cdot) = \|\cdot\|^2$) ensuring smooth optimization.

The target objective defines the desired behavior or goal for the system, while the error computation evaluates deviations from this target. Solving this finite-horizon optimal control problem can be achieved through two major classes of algorithms: direct methods and indirect methods [53]. Predictive sampling is a simple, derivative-free optimization-based direct shooting method used within the MJPC framework. After the target objective defines the desired behavior or goal for the system, while the error computation evaluates deviations from this target, predictive sampling generates multiple control action samples, evaluates their costs, and selects the best path. This process is continuously repeated to adapt to real-time changes (Figure 3). Unlike some complex methods requiring detailed mathematical models and gradient calculations, predictive sampling is straightforward and can be used on less powerful hardware. In the MJPC framework, predictive sampling is one of several available planners.

As MJPC utilizes the MuJoCo physics engine, the non-linearity is handled internally by MuJoCo, and the external disturbances are explicitly modeled within the system dynamics by incorporating additional forces and torques into the equations of motion:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau + J^\top f_{\text{ext}}, \tag{13}$$

where $M(q) \in \mathbb{R}^{18\times 18}$ is the state-dependent mass/inertia matrix, $\ddot{q} \in \mathbb{R}^{18}$ represents the generalized accelerations, $C(q, \dot{q})\dot{q} \in \mathbb{R}^{18}$ captures Coriolis and centrifugal forces, and $G(q) \in \mathbb{R}^{18}$ represents gravitational forces. The control torques applied by the system's actuators are represented by

$\tau \in \mathbb{R}^{18}$, where the first 6 elements correspond to the floating base (always zero) and the remaining 12 elements actuate the joints. The term $J \in \mathbb{R}^{m\times 18}$ is the contact Jacobian (with $J^\top \in \mathbb{R}^{18\times m}$ its transpose) mapping external forces $f_{\text{ext}} \in \mathbb{R}^m$ from Cartesian space to joint space. Here f_{ext} includes both contact forces and external disturbances, with $m = 3 \times$ number of contact points (e.g., $m = 12$ when all four feet are grounded, as each foot contact exerts a 3D force vector).

These dimensions reflect the Unitree Go1's 18 degrees of freedom: 6 for the floating-base (position and orientation in 3D space) plus 12 joint angles (3 joints per leg $\times$ 4 legs). MuJoCo computes $M(q)$, $C(q, \dot{q})$, and $G(q)$ in real-time based on the robot's XML model parameters, including link masses, inertia tensors, and joint properties defined in the simulation setup.

To summarize, in this research, we integrate the Unitree Go1 robot into the extensive pre-existing MJPC package and utilize the predictive sampling method to carry out the benchmarking in the MuJoCo environment, similar to the RL setup. The MPC controller operates in a receding horizon fashion using the predictive sampling planner available in MJPC. At each control step, the current state of the robot—including joint positions, velocities, and actuator states—is used to initialize the planner. A set of candidate control sequences is then generated and forward-simulated using the MuJoCo physics engine to predict future trajectories over a finite horizon. Each trajectory is evaluated using a task-specific cost function based on tracking accuracy and control effort. The control sequence with the lowest cost is selected, and the first action in that sequence is applied to the robot. This process is repeated at every timestep, allowing the controller to adapt continuously to changes in the robot's state and external disturbances.

IV. RESULTS

This section presents the simulation experiments and the results of benchmarking MPC and RL controllers. The benchmarking is conducted on a standardized task to ensure that the controllers are evaluated on the same locomotion task, providing an unbiased and consistent basis for comparison.

A. STANDARDIZED TASK

In this paper, the standardized task chosen for the evaluation involves a straightforward locomotion scenario: walking in a straight line along the x-axis at a constant velocity of 0.5 m/s. Figure 4 shows the results of the standardization task, which ensures that the environments set up in both MPC and RL controllers are on equal grounds concerning straight walking at a constant velocity of 0.5 m/s. This standardization is crucial for justifying further perturbation tests.

The top-left plot illustrates the position of the center of mass of the Go1 robot over time for both the MPC and RL controllers, along with the variance across multiple rollouts for RL controller. It demonstrates that the primary motion occurs along the x-axis, with both controllers enabling

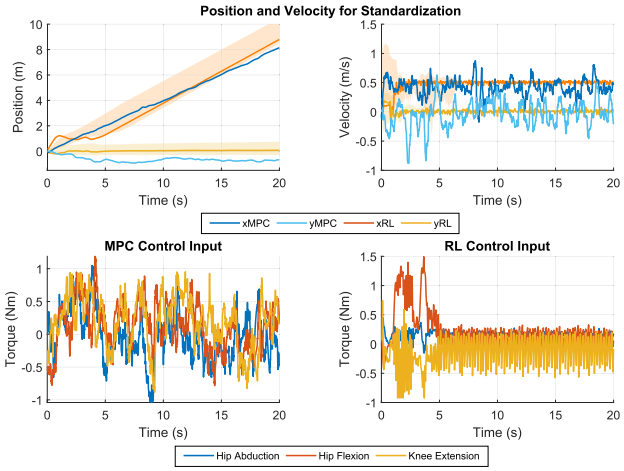


FIGURE 4. The top panels show the Go1 robot's position and velocity under both MPC and RL controllers during the standardized task, while the bottom panels illustrate the corresponding control inputs.

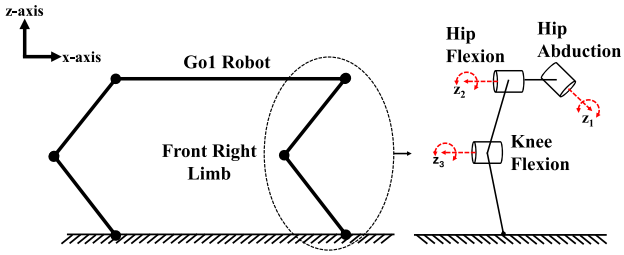


FIGURE 5. Go1 robot single limb configuration.

the robot to walk approximately the same distance within a limited time. The top-right graph displays the velocity data over time for both controllers. In RL, the velocities in the x and y directions stabilize around 0.5 m/s and $0m/s$, respectively, which is represented by the converging variance plot over multiple rollouts of the trained model. This indicates that RL maintains a consistent forward velocity with minimal lateral movement with a variance of 0.0012. The MPC controller also stabilizes around the same velocities but exhibits fluctuations in the y-axis with a variance of 0.056, suggesting that while MPC can achieve and maintain the desired velocity, it requires adjustments to address these fluctuations.

The bottom plots in Figure 4 show the control inputs for a single limb of the quadruped under both controllers. The three curves correspond to the limb's primary joints: hip abduction, hip flexion, and knee extension (see Figure 5). With RL control, it is evident that the knee extension contributes significantly to locomotion, as indicated by its higher amplitude compared to the hip abduction and flexion joints. The higher amplitude indicates that the knee extension joint is predominantly utilized for forward movement. In contrast, the MPC controller achieves locomotion with balanced contributions from all three joints.

B. DISTURBANCE REJECTION CAPABILITY

The disturbance rejection capability is evaluated concerning two metrics: the recovery performance after perturbation

TABLE 3. Quantitative comparison of position deviation, settling time, and maximum allowable disturbance (MAD) for MPC and RL controllers. Values are shown as MPC: RL.

Perturbation Direction	Position Deviation (m)	Settling Time (s)	MAD (N)
Positive x-axis	0.52 : 0.77	0.68 : 0.45	260 : 520
Negative x-axis	0.28 : 0.38	0.66 : 1.35	330 : 950
Negative y-axis	0.78 : 0.60	1.52 : 0.89	340 : 250

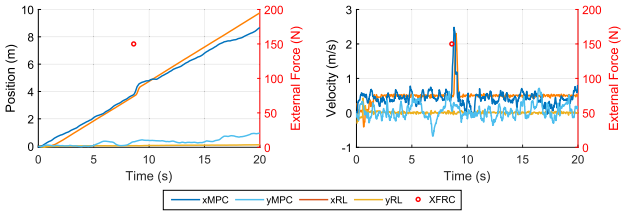


FIGURE 6. Go1 Robot FBD with Perturbation in Positive x-axis (on the left) and Perturbation in Negative x-axis (on the right).

and the maximum allowable disturbance for each controller. To have a detailed comparison, the disturbances are applied at the center of mass (CoM) along various directions. The recovery performance is evaluated based on the overshoot and settling time after perturbation. See Figure 5 for an illustration of the robot and the coordinates.

1) PERTURBATION TESTS DURING LOCOMOTION

We apply perturbation forces to the robot's CoM in various directions while it maintains a constant velocity of 0.5 m/s. The timing of the perturbations is randomly selected but remains consistent for both controllers. These perturbations are handled as external forces in Equation (13). Additionally, perturbation tests with different magnitudes were performed, all yielding similar results. The results presented are from tests with a force of 150N, applied for 0.2 seconds as shown in Table 3.

a: ***PERTURBATION IN POSITIVE*** *X* ***DIRECTION***

Figure 6 illustrates the response of the Go1 robot to the perturbation applied in the positive x-axis (forward direction) during a straight walk at 0.5 m/s, using MPC and RL controllers. The position plots (left column) show that the position under RL control exhibits a smaller deviation (smaller overshoot) of 0.4 in the x-axis than the MPC controller with a deviation of 0.65 just after perturbation. Both controllers can converge the robot to the predesigned constant velocity. The velocity plots (right column) further show that while both controllers experience disturbances at the point of impact, the RL controller stabilizes approximately 0.25s faster (with a shorter settling time), suggesting robustness against perturbation.

In terms of control input, the RL controller significantly relies on the hip flexion joint to compensate for the external force, whereas the MPC controller distributes the effort across all three joints. This distribution indicates that the RL controller focuses on using specific joints to handle disturbances. In contrast, the MPC controller adopts a balanced approach, utilizing multiple joints to manage external forces.

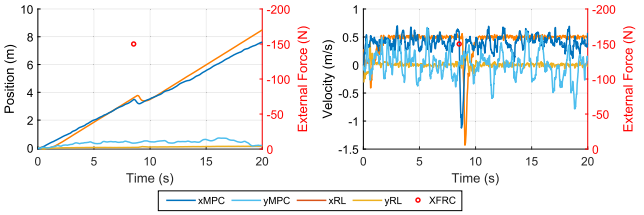


FIGURE 7. Perturbation in negative x-axis.

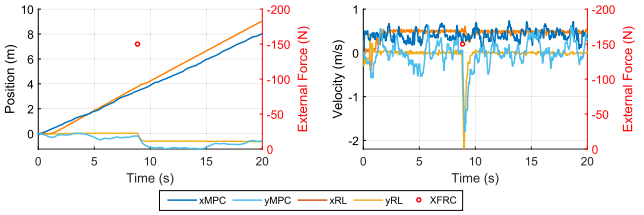


FIGURE 8. Perturbation in negative y-axis.

b: PERTURBATION IN NEGATIVE X DIRECTION

Similarly, the results of a perturbation of 150N along the negative x-axis (opposite to the direction of robot locomotion) while walking at a constant velocity of 0.5 m/s are shown in Figure 7. In the position plots, the RL controller shows a slightly larger deviation in the x-axis position than the MPC controller, taking 0.3s longer to recover noticed in the velocity plot. This is due to the backward force causing the robot to reach an almost failed state, with its base barely touching the ground. As aforementioned, the RL controller relies primarily on a single joint, resulting in a slower recovery. Our simulation further shows that the RL controller heavily relies on the hip flexion joint to compensate for the external force along the x direction.

c: PERTURBATION IN Y DIRECTION

The quadrupedal robot's left and right sides are symmetric, so if the gait were perfectly symmetric, the responses to perturbations in the positive and negative y-axis would be identical. However, due to slight asymmetry in the gait, the responses are not identical but remain very similar. Figure 8 shows the robot's response to the perturbation in the negative y-axis while moving in the positive x direction.

In the velocity plots (right column), the velocity under the MPC controller shows a smaller deviation (larger overshoot) in the y-axis than the RL controller just after perturbation. Despite this, the robot under RL control demonstrates a 0.33s shorter recovery time, which can be observed from both the position and velocity plots. This suggests that the robot under the RL control is robust enough to reject perturbations along the y-axis. Similar behavior is also observed for perturbations in the positive y-axis due to the robot's symmetry.

2) MAXIMUM ALLOWABLE DISTURBANCE

We define failure as the point at which the robot is unable to maintain balance across its four-foot contact points, resulting

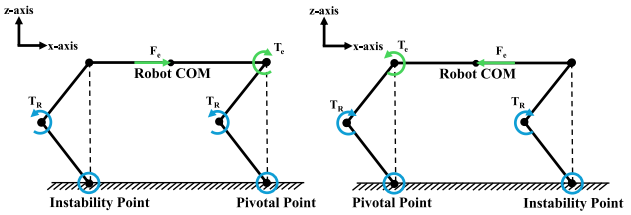


FIGURE 9. Free body diagram of a quadrupedal robot under perturbation.

in other parts of the robot, such as the torso, touching the ground. This loss of balance leading to the robot body contacting the ground is a failure, irrespective of the recovery. The maximum perturbation values for both the RL and MPC controllers are determined through a systematic process. External forces were applied to the robot in both the x and y directions, and their magnitudes were incrementally increased in steps of $10N$. At each increment, the robot's response was observed to check whether the defined failure condition was triggered. The largest force for which the robot remained stable across all trials was recorded as the maximum allowable disturbance. The applied perturbation follows a triangular force profile, peaking over a short duration of 0.2 seconds. As a result, although the maximum force can reach f N, the total impulse delivered to the robot is less than that of a step force with the same peak magnitude f N. The impulse is calculated as: $0.5 \times \text{duration} \times f$.

By using the RL control, the Go1 robot can withstand a maximum perturbation force of $520N$ in the positive x-axis and $950N$ in the negative x-axis. Along the y-axis, the RL model handles up to $210N$ in the positive direction and $250N$ in the negative direction, beyond which the robot fails to recover locomotion. When tested with the MPC controller, the robot fails at $260N$ in the positive x-axis and $330N$ in the negative x-axis. Along the y-axis, failure occurs at $290N$ in the positive direction and $340N$ in the negative direction. The MPC controller can reject larger disturbances along the y-axis, partly because it controls all three joints in a balanced manner. In contrast, the trained RL control policy relies heavily on a single joint, the knee extension, which contributes to the forward locomotion. The observed asymmetry in x-direction disturbance rejection is due to the robot's dynamic response to directional torque. Perturbations in the negative x-direction (opposite to motion) induce torque that presses the feet into the ground, reinforcing contact and stability. Conversely, positive x-direction forces can lift the feet, reducing ground contact and destabilizing the robot.

In summary, the RL controller exhibits disturbance rejection capabilities characterized by transient performance metrics such as reduced overshoot and accelerated response following perturbations. It is capable of rejecting disturbances of greater magnitude along the x-axis. However, due to the trained RL control policy heavily relying on one joint, the knee extension, it cannot reject as large a disturbance along the y-axis as the MPC controller does.

Also note that for both controllers, the robot can handle significantly larger disturbance in the negative x-axis than the positive direction. This increased rejection in the negative x-axis is due to the robot's ability to maintain ground contact with all four feet under backward perturbation force. This is explained in Figure 9, which shows the side view of the Go1 robot with an external force F_e acting at the CoM on the torso in positive and negative directions, respectively, generating torque T_e. The robot counters the perturbation torque with torque T_R. In Figure 9, on the left, the torque T_R causes the robot's foot to lift off the ground, leading to instability. In contrast, Figure 9, on the right, shows that the torque tends to have the robot push the feet to the ground, thus increasing stability around the pivotal point and maintaining balance.

C. ENERGY EFFICIENCY

The energy efficiency of each control strategy is assessed based on the Cost of Transport (CoT) [54]. CoT measures the energy required to move a system relative to its weight and speed, providing a standard metric for energy efficiency in locomotion,

$$\text{CoT} = \frac{\text{Power Input}}{\text{Total Weight} \times \text{Speed}} = \frac{P}{mgv} \text{ [55]} \tag{14}$$

Here, the power input P is computed as the average power throughout the simulation T,

$$P = \frac{1}{T}\int_0^T \sum_i \tau_i \dot{\theta} dt, \tag{15}$$

where τ_i is the Torque at the ith timestep and $\dot{\theta}$ is the angular velocity.

The calculations of CoT varies among authors, depending on the assumptions regarding the energy absorption and recovery [56], [57]. When calculating the total power P, this work sets negative values as zero, which is based on the assumption that the energy recovery is either inefficient or not a significant contributor to the system's net energy cost. This offers a simpler approach and more interpretable calculation of CoT, particularly in cases where energy recovery is not explicitly modeled. Specifically, the formulation of CoT is shown in Equation (16):

$$\text{CoT} = \frac{\sum_{i=1}^{T}\sum_{j=1}^{n} E_{i,j}}{mgd} \tag{16}$$

$$E_{i,j} = \begin{cases} F^{\text{a}}_{i,j} \cdot v_{i,j} \cdot \Delta t, & \text{if } F^{\text{a}}_{i,j} \cdot v_{i,j} > 0, \\ 0, & \text{otherwise} \end{cases} \tag{17}$$

In Equation (16), n represents the total number of joints in the system. The term $F^{\text{a}}_{i,j}$ refers to the force exerted by the actuator at the ith timestep for j^{th} joint, while the $v_{i,j}$ denotes the angular velocity at the ith timestep for the corresponding joint. The parameter Δt is the timestep used in the simulation, $m = 12.0kg$ is the total mass of the robot, and $g = 9.81m/s^2$ is the gravity acceleration. The final computed CoT for RL and MPC are 1.768 and 2.993, respectively, measured over a constant time $t = 50\ sec$. This indicates that

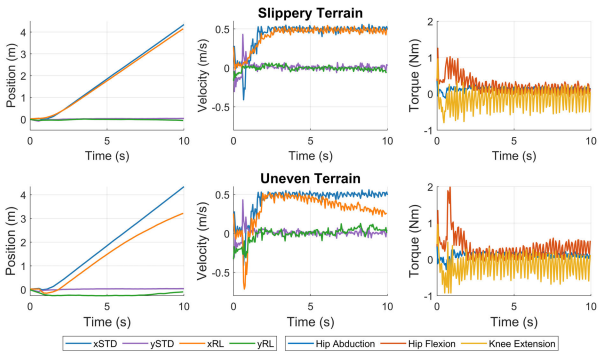


FIGURE 10. Generalization of slippery and uneven terrain when compared with the standardized task.

the RL controller, focusing on specific joints, can achieve a lower CoT compared to the MPC controller, suggesting an energy-efficient locomotion strategy.

V. DISCUSSION

This study highlights significant differences between the MPC and RL controllers in terms of robustness and energy efficiency. The trained RL controller exhibits generally larger disturbance rejection capabilities with reduced energy consumption requirements. In contrast, the MPC controller achieves a balanced distribution of control efforts across joints, which contributes to its ability to recover from disturbances and maintain stable locomotion.

A. FAILURE AND RECOVERY

One noteworthy observation is the contrasting failure and recovery mechanisms between the two controllers. The RL controller's reliance on specific joints for locomotion poses a challenge when the robot encounters failure, such as when the torso makes contact with the ground. In these scenarios, our experiments show that the RL controller struggles to recover and continue moving. On the other hand, the MPC controller, which evenly distributes control efforts across all joints, can recover from such failures and resume walking. This distinction underscores the importance of joint utilization strategies in ensuring robust recovery capabilities.

B. RL GENERALIZATION

In this study, we have further evaluated the generalization [58] performance of the RL controller across different terrains, specifically focusing on slippery flat surfaces and uneven terrains. The results reveal significant challenges for the RL controller when encountering environments vastly different from its training setting.

When testing on a slippery surface, the RL controller, initially trained on flat, frictioned terrain, exhibits a noticeable decrease in the distance traveled within the same time frame (Figure 10). This reduction occurs primarily due to the compromised movement efficiency from lack of friction. The robot struggles to reach the target velocity of 0.5 m/s, with a slower acceleration phase reflecting the reduced traction. The controller's control signals are variable, indicating its efforts to adapt to the slippery conditions and maintain stability. Similarly, the performance on uneven terrain demonstrates difficulty in maintaining consistent movement. The robot travels an even shorter distance on the uneven surface (Figure 10), and the fluctuations in the x and y positions highlight the challenge of sustaining a steady trajectory. The velocity plots also show significant variations in the x and y directions, indicating the robot's struggle to maintain a constant velocity of 0.5 m/s amidst varying elevations. The control signals show increased activity in the hip flexion and knee extension joints as the controller attempts to compensate for the terrain irregularities.

These findings show the ability of the RL controller to generalize to new environments, emphasizing the need for specialized training techniques or hybrid approaches to enhance robustness and adaptability. This highlights the importance of continued research into improving the generalization capabilities of RL-based controllers for real-world applications.

VI. CONCLUSION

In conclusion, this paper presents a detailed comparison between MPC and RL methods for legged robot locomotion. The results indicate that RL demonstrates larger disturbance rejection, with 0.25s to 0.33s shorter settling time after perturbations, and improved energy efficiency, with a CoT 1.23 lower than that of MPC. This improvement is partly attributed to RL's high-frequency control inputs. In contrast, MPC provides enhanced stability and recovery performance when subjected to large perturbations. Additionally, RL's performance decreases notably on slippery and uneven terrains, highlighting the need for improved generalization capabilities in practical applications. Under both controllers, the robot demonstrates larger disturbance rejection in the negative x-axis than in the positive direction. When perturbed along the positive x-axis, the balance torque generated by the knee joints causes the robot's foot to lift off the ground, resulting in instability. Beyond this comparison, this work establishes robust simulation environments for the Go1 robot in MuJoCo, with implementations of both RL and MPC approaches. To summarize, this work provides clear insights into the practical trade-offs between MPC and RL for quadruped locomotion, offering guidance on controller selection based on task requirements. By standardizing the benchmarking environment and making the code and setup public, this study also serves as a valuable resource for new researchers entering the field.

Despite promising results in simulation, DRL remains highly susceptible to the sim-to-real gap, which arises from the inherent differences between simulated environments and the complexities of the real world [59], [60]. A key direction for future research is validating these approaches in real-world scenarios and adopting strategies to bridge the sim-to-real gap, which is caused by practical factors such as time delay, contact modeling errors, and sensor

noise when deploying the controller on hardware. Techniques such as domain randomization, which introduce variability in simulated parameters like mass, friction, and terrain properties, can improve generalization [61], [62]. Furthermore, system identification methods can calibrate simulation models to closely align with the physical robot's dynamics, thereby minimizing discrepancies between simulation and reality [63]. While this study focuses on simulation-based benchmarking, allowing for reproducible and controlled evaluations, we acknowledge the importance of validating the controllers on physical hardware. Thus, a major direction for future work will involve deploying both the MPC and RL controllers on the Unitree Go1 robot hardware, addressing sim-to-real challenges such as sensor noise, unmodeled dynamics, computational load, and real-time performance issues.

Another direction for future work involves adding structured gait constraints or specifications, and expanding the fundamental locomotion control task to encompass a broader range of tasks, including turning, obstacle avoidance and testing with more advanced MPC and RL controllers. This includes evaluating RL algorithms such as SAC and TD3, as well as exploring hybrid approaches that integrate MPC and RL to enhance robustness and adaptability.

ACKNOWLEDGMENT

The authors would like to thank Dr. Zherong Pan for the meaningful discussions and his help in setting up the simulation environments. OpenAI's GPT-3 and GPT-4 models were also used for assistance in language checks in this document.

REFERENCES

[1] S. Kajita and B. Espiau, "Legged robot," in *Springer Handbook of Robotics*. Cham, Switzerland: Springer, 2008, pp. 361–389.

[2] R. Siegwart, M. Hutter, P. Oettershagen, M. Burri, I. Gilitschenski, E. Galceran, and J. Nieto, "Legged and flying robots for disaster response," in *Proc. World Eng. Conf. Conv. (WECC)*. ETH-Zürich, Jan. 2015.

[3] H. Kolvenbach, E. Hampp, P. Barton, R. Zenkl, and M. Hutter, "Towards jumping locomotion for quadruped robots on the moon," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Nov. 2019, pp. 5459–5466.

[4] R. B. Mcghee and G. I. Iswandhi, "Adaptive locomotion of a multilegged robot over rough terrain," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-9, no. 4, pp. 176–182, Apr. 1979.

[5] S. H. Seok, A. Wang, M. Y. Chuah, D. Otten, J. Lang, and S. J. Kim, "Design principles for highly efficient quadrupeds and implementation on the MIT cheetah robot," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2013, pp. 3307–3312.

[6] M. Hutter, C. Gehring, D. Jud, A. Lauber, C. D. Bellicoso, V. Tsounis, J. Hwangbo, K. Bodie, P. Fankhauser, M. Bloesch, R. Diethelm, S. Bachmann, A. Melzer, and M. Hoepflinger, "ANYmal–a highly mobile and dynamic quadrupedal robot," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2016, pp. 38–44.

[7] I. Poulakakis, J. A. Smith, and M. Buehler, "Modeling and experiments of untethered quadrupedal running with a bounding gait: The scout II robot," *Int. J. Robot. Res.*, vol. 24, no. 4, pp. 239–256, Apr. 2005.

[8] A. J. Ijspeert, "Central pattern generators for locomotion control in animals and robots: A review," *Neural Netw.*, vol. 21, no. 4, pp. 642–653, May 2008. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0893608008000804

[9] S. H. Zak, "An introduction to model-based predictive control (MPC)," Lecture Notes ECE680, Purdue Univ., 2017.

[10] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *Int. J. Robot. Res.*, vol. 32, no. 11, pp. 1238–1274, Sep. 2013.

[11] Y. Yang, K. Caluwaerts, A. Iscen, T. Zhang, J. Tan, and V. Sindhwani, "Data efficient reinforcement learning for legged robots," in *Proc. Conf. Robot Learn.*, Jan. 2019, pp. 1–10. [Online]. Available: https://proceedings.mlr.press/v100/yang20a.html

[12] S. Gangapurwala, M. Geisert, R. Orsolino, M. Fallon, and I. Havoutis, "RLOC: Terrain-aware legged locomotion using reinforcement learning and optimal control," *IEEE Trans. Robot.*, vol. 38, no. 5, pp. 2908–2927, Oct. 2022.

[13] N. Rathod, A. Bratta, M. Focchi, M. Zanon, O. Villarreal, C. Semini, and A. Bemporad, "Model predictive control with environment adaptation for legged locomotion," *IEEE Access*, vol. 9, pp. 145710–145727, 2021.

[14] R. Grandia, F. Farshidian, R. Ranftl, and M. Hutter, "Feedback MPC for torque-controlled legged robots," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Nov. 2019, pp. 4730–4737.

[15] Y. Song, A. Romero, M. Müller, V. Koltun, and D. Scaramuzza, "Reaching the limit in autonomous racing: Optimal control versus reinforcement learning," *Sci. Robot.*, vol. 8, no. 82, p. 1462, Sep. 2023.

[16] Unitree. (2022). *Go1*. Accessed: Jul. 8, 2024. [Online]. Available: https://www.unitree.com/go1/

[17] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*, vol. 1, no. 1. Cambridge, U.K.: MIT Press, 1998.

[18] C. J. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, nos. 3–4, pp. 279–292, May 1992.

[19] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing Atari with deep reinforcement learning," 2013, *arXiv:1312.5602*.

[20] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Sep. 2018.

[21] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.

[22] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine, "Soft actor-critic algorithms and applications," 2018, *arXiv:1812.05905*.

[23] J. Achiam, "Spinning up in deep reinforcement learning," 2018.

[24] G. B. Margolis, G. Yang, K. Paigwar, T. Chen, and P. Agrawal, "Rapid locomotion via reinforcement learning," *Int. J. Robot. Res.*, vol. 43, no. 4, pp. 572–587, Apr. 2024.

[25] H. Kimura and S. Kobayashi, "Reinforcement learning for locomotion of a two-linked robot arm," in *Proc. 6th Eur. Workshop Learn. Robots*, 1997, pp. 144–153.

[26] S. Gu, E. Holly, T. Lillicrap, and S. Levine, "Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2017, pp. 3389–3396.

[27] S. Jeon, M. Jung, S. Choi, B. Kim, and J. Hwangbo, "Learning whole-body manipulation for quadrupedal robot," *IEEE Robot. Autom. Lett.*, vol. 9, no. 1, pp. 699–706, Jan. 2024.

[28] M. A. Henson, "Nonlinear model predictive control: Current status and future directions," *Comput. Chem. Eng.*, vol. 23, no. 2, pp. 187–202, Dec. 1998.

[29] A. Bemporad and M. Morari, "Robust model predictive control: A survey," in *Lecture Notes in Control and Information Sciences*, 2007, pp. 207–226.

[30] M. Schwenzer, M. Ay, T. Bergs, and D. Abel, "Review on model predictive control: An engineering perspective," *Int. J. Adv. Manuf. Technol.*, vol. 117, nos. 5–6, pp. 1327–1349, Aug. 2021.

[31] S. J. Qin and T. A. Badgwell, "A survey of industrial model predictive control technology," *Control Eng. Pract.*, vol. 11, no. 7, pp. 733–764, Jul. 2003.

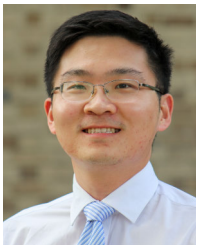
[32] L. Singh and J. Fuller, "Trajectory generation for a UAV in urban terrain, using nonlinear MPC," in *Proc. Amer. Control Conf.*, vol. 3, 2001, pp. 2301–2308.

[33] F. Farshidian, E. Jelavic, A. Satapathy, M. Giftthaler, and J. Buchli, "Real-time motion planning of legged robots: A model predictive control approach," in *Proc. IEEE-RAS 17th Int. Conf. Humanoid Robot. (Humanoids)*, Nov. 2017, pp. 577–584.

[34] J. Di Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim, "Dynamic locomotion in the MIT cheetah 3 through convex model-predictive control," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2018, pp. 1–9.
[35] M. Gaertner, M. Bjelonic, F. Farshidian, and M. Hutter, "Collision-free MPC for legged robots in static and dynamic scenes," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2021, pp. 8266–8272.
[36] S. Ha, J. Lee, M. van de Panne, Z. Xie, W. Yu, and M. Khadiv, "Learning-based legged locomotion; state of the art and future perspectives," 2024, *arXiv:2406.01152*.
[37] G. Bellegarda and K. Byl, "An online training method for augmenting MPC with deep reinforcement learning," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2020, pp. 5453–5459.
[38] Z. Zhang, X. Chang, H. Ma, H. An, and L. Lang, "Model predictive control of quadruped robot based on reinforcement learning," *Appl. Sci.*, vol. 13, no. 1, p. 154, Dec. 2022. [Online]. Available: https://www.mdpi.com/2076-3417/13/1/154
[39] Y. Chen and Q. Nguyen, "Learning agile locomotion and adaptive behaviors via RL-augmented MPC," 2023, *arXiv:2310.09442*.
[40] O. Tutsoy, D. Erol Barkana, and S. Colak, "Learning to balance an NAO robot using reinforcement learning with symbolic inverse kinematic," *Trans. Inst. Meas. Control*, vol. 39, no. 11, pp. 1735–1748, Nov. 2017.
[41] A. Hartmann, D. Kang, F. Zargarbashi, M. Zamora, and S. Coros, "Deep compliant control for legged robots," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2024, pp. 11421–11427.
[42] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Oct. 2012, pp. 5026–5033.
[43] D. Blanco-Mulero, O. Barbany, G. Alcan, A. Colomé, C. Torras, and V. Kyrki, "Benchmarking the Sim-to-Real gap in cloth manipulation," *IEEE Robot. Autom. Lett.*, vol. 9, no. 3, pp. 2981–2988, Mar. 2024.
[44] S. E. Ada, E. Ugur, and H. L. Akin, "Generalization in transfer learning: Robust control of robot locomotion," *Robotica*, vol. 40, no. 11, pp. 3811–3836, Nov. 2022.
[45] H. Wang, V. Caggiano, G. Durandau, M. Sartori, and V. Kumar, "MyoSim: Fast and physiologically realistic MuJoCo models for musculoskeletal and exoskeletal studies," in *Proc. Int. Conf. Robot. Autom. (ICRA)*, May 2022, pp. 8104–8111.
[46] S. Akki. (2024). *Benchmarking Model Predictive Control and Reinforcement Learning Based Control for Legged Robot Locomotion*. [Online]. Available: https://github.com/RoLACLab/mpc-and-rl-benchmark
[47] J. Schulman, S. Levine, P. Abbeel, M. I. Jordan, and P. Moritz, "Trust region policy optimization," in *Proc. 32nd Int. Conf. Mach. Learn.*, Jordan, Jul. 2015, pp. 1889–1897. [Online]. Available: https://proceedings.mlr.press/v37/schulman15.html
[48] J. W. Mock and S. S. Muknahallipatna, "A comparison of PPO, TD3 and SAC reinforcement algorithms for quadruped walking gait generation," *J. Intell. Learn. Syst. Appl.*, vol. 15, no. 1, pp. 36–56, 2023.
[49] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "Openai gym," 2016, *arXiv:1606.01540*.
[50] A. Raffin. (2020). *Rl Baselines3 Zoo*. [Online]. Available: https://github.com/DLR-RM/rl-baselines3-zoo
[51] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2019, pp. 2623–2631.
[52] T. Howell, N. Gileadi, S. Tunyasuvunakool, K. Zakka, T. Erez, and Y. Tassa, "Predictive sampling: Real-time behaviour synthesis with mujoco," 2022, *arXiv:2212.00541*.
[53] M. Kelly, "An introduction to trajectory optimization: How to do your own direct collocation," *SIAM Rev.*, vol. 59, no. 4, pp. 849–904, Jan. 2017.
[54] T. Chen, J. P. Schmiedeler, and B. Goodwine, "Robustness and efficiency insights from a mechanical coupling metric for ankle-actuated biped robots," *Auto. Robots*, vol. 44, no. 2, pp. 281–295, Jan. 2020.
[55] S. Kim and P. M. Wensing, "Design of dynamic legged robots," *Found. Trends Robot.*, vol. 5, no. 2, pp. 117–190, 2017.
[56] A. J. Ijspeert, "Biorobotics: Using robots to emulate and investigate agile locomotion," *Science*, vol. 346, no. 6206, pp. 196–203, Oct. 2014.
[57] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Sci. Robot.*, vol. 5, no. 47, p. 5986, Oct. 2020.
[58] D. Ghosh, J. Rahme, A. Kumar, A. Zhang, R. P. Adams, and S. Levine, "Why generalization in RL is difficult: Epistemic POMDPs and implicit partial observability," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2021, pp. 25502–25515.
[59] P. Akella, W. Ubellacker, and A. D. Ames, "Test and evaluation of quadrupedal walking gaits through Sim2Real gap quantification," 2022, *arXiv:2201.01323*.
[60] C. S. Imai, M. Zhang, Y. Zhang, M. Kierebinski, R. Yang, Y. Qin, and X. Wang, "Vision-guided quadrupedal locomotion in the wild with multi-modal delay randomization," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2022, pp. 5556–5563.
[61] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-Real transfer of robotic control with dynamics randomization," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2018, pp. 3803–3810.
[62] Z. Li, X. Cheng, X. B. Peng, P. Abbeel, S. Levine, G. Berseth, and K. Sreenath, "Reinforcement learning for robust parameterized locomotion control of bipedal robots," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2021, pp. 2811–2817.
[63] W. Yu, J. Tan, C. Karen Liu, and G. Turk, "Preparing for the unknown: Learning a universal policy with online system identification," 2017, *arXiv:1702.02453*.



SHIVAYOGI AKKI (Student Member, IEEE) received the B.E. degree in mechanical engineering from Visvesvaraya Technological University, Karnataka, India, in 2021, and the M.S. degree in mechatronics, robotics and automation engineering from Michigan Technological University, Michigan, USA, in 2023, where he is currently pursuing the Ph.D. degree in electrical and computer engineering. His research interests lie in the fields of robotics, machine learning, and control systems.



TAN CHEN (Member, IEEE) received the B.S. degree in mechanical engineering from Shanghai Jiao Tong University, China, in 2013, the dual M.Eng. degree in optimization and automation of industrial processes from Ecole des Mines de Douai, France, in 2014, supported by the Eiffel Scholarship, the M.S. degree in mechanical engineering from Shanghai Jiao Tong University in 2016, and the Ph.D. degree in aerospace and mechanical engineering from the University of Notre Dame, Notre Dame, IN, USA, in 2021. From 2021 to 2022, he was a Post-Doctoral Research Associate with the Coordinated Science Laboratory, University of Illinois Urbana-Champaign. He is currently an Assistant Professor of electrical and computer engineering with Michigan Technological University. His research interests lie in robotics, dynamical systems, and nonlinear control, with a focus on legged robots.